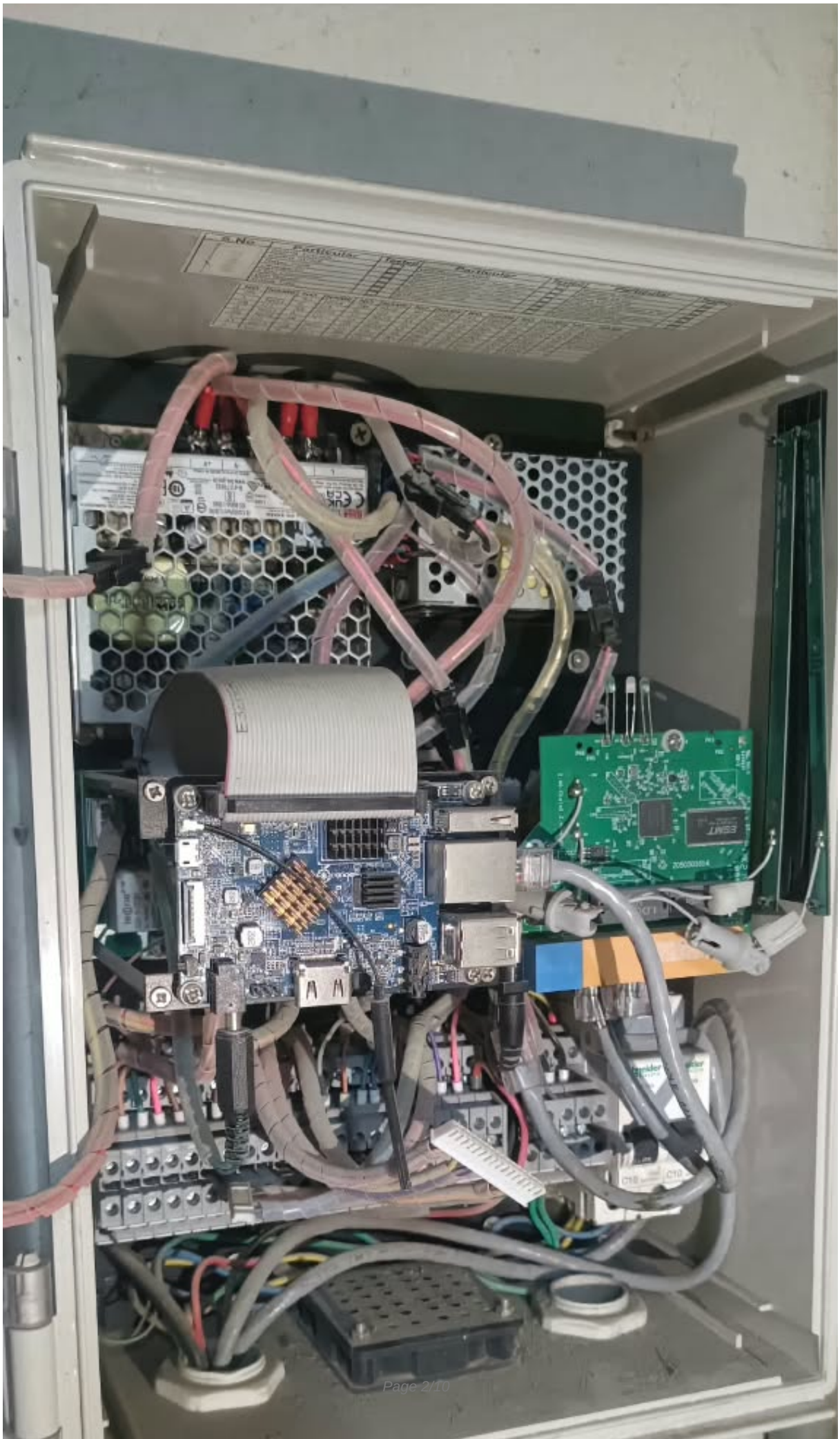
GateAccess MVP

Complete Beginner Setup Guide

How to add an ESP32 (Arduino) or Raspberry Pi to your gate box, upload/run the controller, connect MQTT, and control the gate from the React Native phone app.



Your control enclosure (image.jpeg)

1. What this project does (simple words)

You have a control box that opens a gate boom barrier. Today that box already has power, wiring, and an old computer board (the blue board in the photo). We will add a new small controller board as the brain. For MVP we recommend ESP32 (Arduino, low cost). Raspberry Pi is optional for future camera work. The controller will:

- Read RFID chips (tags) with an RFID scanner.
- If the chip is allowed, open the gate for about 1 second using a relay.
- Talk to your phone app over the internet using MQTT messages.
- Also open the gate when you press Remote Unlock in the app.

MVP means Minimum Viable Product: make it work first. The UI does not need to look fancy.

IMPORTANT

You are NOT flashing firmware into the blue/green boards in the photo. You ADD an ESP32 or Raspberry Pi, upload/run our scripts on THAT board, and rewire RFID + gate open signal to it. Backup the blue board first.

1.1 Parts of the system

Control box	Enclosure in your photo - power, boards, terminals
ESP32 (Arduino)	Recommended MVP - Wi-Fi, ~300-800 INR board
Raspberry Pi	Optional - Python, better for future camera
RFID reader	Module that reads chip UIDs over SPI
RFID chips/tags	Cards/keyfobs people tap to enter
Relay	Electronic switch that pulses the boom open
MQTT broker	Public message post office: test.mosquitto.org
Phone app	React Native app installed on your Android phone

2. Looking at your box (image.jpeg)

Match these labels to what you see inside the open enclosure:

Top - Power Supply Unit (PSU)

Large metal unit with a honeycomb/mesh cover. Thick wires leave its terminals. This powers the whole system. Keep it. The ESP32/Pi will take 5V from here (USB adapter, or buck converter from 12V/24V if you know how to wire safely).

Center - Blue board with gold heatsink

This is the CURRENT brain (old single-board computer). You may see USB ports, a power jack, Ethernet, and a ribbon cable. Do NOT delete its software yet. First back it up. Later the ESP32 or Pi replaces its job for our MVP.

Right - Green board

Secondary / interface board. Often has network (Ethernet cable plugged in). Usually leave it alone unless RFID or relay is wired only through it.

Middle - Gray terminal blocks (DIN rail)

Long row of screw terminals. Field cables (boom open signal, sensors, ground) land here. This is the MOST IMPORTANT area for wiring: connect the new controller so it drives the same open-gate path the old board used.

Bottom - Cable glands

Thick cables leave the box to the boom and possibly an external RFID reader. A good area to mount the ESP32 or Pi (not under the hot PSU).

3. How MQTT works here (for beginners)

MQTT is a simple messaging system. Think of a post office (the broker). The controller (ESP32 or Pi) and the phone do not call each other directly. Both connect to the same broker and leave/pick up messages on named topics (mailboxes).

Broker we use for MVP testing

Host: test.mosquitto.org (public test broker - fine for learning, not for production secrets).

- ESP32 / Raspberry Pi connect with TCP on port 1883
- Phone app connects with WebSocket: ws://test.mosquitto.org:8080

Both still share the SAME topics, so they can talk through that broker.

Topics (the mailboxes)

gate/cmd	Phone publishes unlock. Example: {"action":"unlock"}
gate/access	Controller publishes GRANTED, DENIED, or REMOTE_UNLOCK

Flow A - Someone taps an authorized RFID chip

```
RFID chip -> RFID reader (SPI) -> ESP32 or Pi controller
  -> checks AUTHORIZED_TAGS (config.h or config.py)
  -> turns relay ON for 1 second (boom opens)
  -> publishes gate/access = GRANTED
  -> phone app log shows GRANTED
```

Flow B - You press Remote Unlock in the phone app

```
Phone app -> publishes gate/cmd {"action":"unlock"}
  -> broker delivers message to controller
  -> controller turns relay ON for 1 second
  -> publishes gate/access = REMOTE_UNLOCK
  -> phone log shows REMOTE_UNLOCK
```

IMPORTANT

The boom itself is NOT opened over MQTT. MQTT only carries messages. Opening the gate is always local wiring: GPIO -> relay -> terminals -> boom.

Where is the app hosted?

For this MVP the React Native app is NOT a website on a server. You install it on an Android phone. It only needs internet so it can reach the MQTT broker. The ESP32 or Pi is the controller in/near the box. Later you can distribute the APK via Play Store or an internal link - that is app distribution, not gate-control hosting.

4. What to do BEFORE you touch wires

1. Take clear photos of every cable and terminal label (door legend too).
2. Label wires with tape if labels are missing.
3. Backup old scripts from the blue board (USB copy / SSH / disk image).
4. Power OFF the enclosure before changing wiring.
5. Confirm boom open voltage/safety with someone who knows the site if unsure.
6. Have ready: ESP32 dev board (recommended) OR Raspberry Pi kit, 5V power, RFID module, relay module (if not already present), Wi-Fi with internet.

IMPORTANT

If you are unsure which terminal opens the boom, do NOT guess with live power. Trace from the old board relay output to the terminals while powered down.

5. Where to inject the controller (ESP32 or Pi)

5.1 Physical placement

- Mount the board in free space (often near the bottom cable area).
- Use standoffs so it does not short against metal.
- Keep it away from the hot PSU at the top.
- Route cables neatly; leave slack for future service.

5.2 Power inject

- ESP32: often 5V via USB phone charger or regulated 5V/GND.
- Pi: stable 5V with enough current; buck converter from 12V/24V if needed.
- Share common ground (GND) between controller, relay module, and existing logic.

5.3 Network inject

- ESP32: Wi-Fi only - set SSID/password in config.h; needs internet for MQTT.
- Pi: Ethernet or Wi-Fi; test: ping test.mosquitto.org

5.4 RFID inject (MFRC522 SPI)

- Document then disconnect RFID SPI from the old blue board.
- Wire RFID to the new board (3.3V logic only).
- ESP32 default pins: SS=5, RST=22, SCK/MOSI/MISO as in esp32 README.
- Pi: enable SPI via raspi-config.

5.5 Gate / boom inject (most important)

Find how the OLD system opens the boom (relay input or dry contact into terminals). Move that CONTROL INPUT to the new controller:

```
ESP32 GPIO 26 (default) --> Relay IN
Pi GPIO BCM 17 (default) --> Relay IN
Relay COM/NO --> same boom-open terminals the old board used
```

ESP32: edit RELAY_PIN in gate_access/config.h. Pi: edit RELAY_PIN in config.py.

5.6 ESP32 wiring quick table (defaults in our sketch)

MFRC522 SDA	ESP32 GPIO 5
MFRC522 RST	ESP32 GPIO 22
Relay IN	ESP32 GPIO 26
MFRC522 power	3.3V and GND (not 5V on ESP32 GPIO)

6. Software: scripts from start (step by step)

6.1 Files you care about

gate_access.ino	ESP32 main sketch (recommended)
config.h (ESP32)	Wi-Fi, MQTT, pins, AUTHORIZED_TAGS
main.py	Pi program (optional)
config.py (Pi)	Broker, topics, GPIO, tags
react-native-app/	Phone app (App.js + mqttService.js)
TESTING.md / README.md	Checklists and quick links

6.2 Desk test first (no enclosure needed)

On your laptop, from the GateAccess folder:

```
bash scripts/verify-all.sh

# Optional: simulate controller without GPIO/RFID
cd raspberry-pi-controller
python3 -m venv .venv
.venv/bin/pip install -r requirements-smoke.txt
.venv/bin/python main.py --sim
```

In --sim mode you can type a tag UID. Sample authorized IDs in config.py are 123456789 and 987654321 - replace these with real chip IDs later.

6.3 ESP32 (Arduino IDE) - recommended

1. Install Arduino IDE + ESP32 board support (Espressif).
2. Library Manager: install MFRC522 and PubSubClient.
3. Open esp32-arduino-controller/gate_access/gate_access.ino
4. Edit config.h: WIFI_SSID, WIFI_PASSWORD, pins if needed.
5. Connect ESP32 by USB, select ESP32 Dev Module, click Upload.
6. Serial Monitor 115200: WiFi OK, MQTT connected, Waiting for RFID tag...

Tap each chip; copy Tag Detected numbers into AUTHORIZED_TAGS in config.h; upload again.

6.4 Raspberry Pi (optional)

1. Flash Raspberry Pi OS, enable SPI, copy raspberry-pi-controller/ to Pi.
2. pip install -r requirements.txt; edit config.py; run python3 main.py.

6.5 Learn RFID chip numbers

ESP32: Serial Monitor line Tag Detected. Pi: console Tag Detected. Use the number from the controller you actually run (they may differ slightly).

6.6 Phone app (React Native)

The react-native-app folder is APP SOURCE. Create a React Native Android project shell once, then copy our files in:

```
npx @react-native-community/cli@latest init GateAccessApp --version 0.73.0
# Copy into that project: App.js, index.js, src/
# Merge package.json dependencies from our package.json
# In android/app/src/main/AndroidManifest.xml add:
#   INTERNET permission
```

```
#   android:usesCleartextTraffic="true" on <application>
npm install
npm run android
```

When the app opens: MQTT should become ONLINE. Press Remote Unlock. Controller should show remote unlock and the relay should click / boom pulse.

7. Full inject day checklist (on the real machine)

1. Power OFF enclosure. Photograph wiring.
2. Mount ESP32 (or Pi) + 5V power + Wi-Fi/network.
3. Wire RFID SPI to the new controller.
4. Wire relay IN to GPIO (ESP32: 26 default, Pi: 17 default).
5. Upload sketch (ESP32) or run main.py (Pi).
6. Capture tag UIDs into config.h or config.py; re-upload/restart.
7. Tap authorized tag: boom opens about 1s; app shows GRANTED.
8. Press Remote Unlock on phone: boom opens; app shows REMOTE_UNLOCK.
9. Tap unauthorized tag: DENIED; boom stays closed.
10. Only after this works, decide whether to retire old blue-board software.

8. Common problems

ESP32 WiFi fails	SSID/password in config.h; 2.4 GHz Wi-Fi; signal in box
ESP32 MQTT fails	Internet on Wi-Fi; broker test.mosquitto.org:1883
Pi cannot connect MQTT	Internet? ping test.mosquitto.org
App stays OFFLINE	Phone internet; cleartext traffic; WS port 8080
No RFID reads	SPI enabled? 3.3V? Wiring MOSI/MISO/SCK/SS/RST?
Tag always DENIED	UID missing from AUTHORIZED_TAGS (exact number)
MQTT works, boom silent	Relay wiring / wrong terminal / GPIO pin mismatch
Relay chatters or sticks	Power/GND issues; relay may need separate supply
Public broker slow/flaky	Retry; later use your own Mosquitto server

9. Safety notes

- Gate barriers can injure people - keep the area clear while testing.
- Isolate mains voltage; as a beginner, work only on low-voltage control signals.
- test.mosquitto.org is public. For real sites use a private broker with authentication.
- Do not leave the enclosure open outdoors; reseal glands and door.

10. Quick reference

```
Controller: test.mosquitto.org:1883 (ESP32 + Pi)
App:        ws://test.mosquitto.org:8080
Topics:     gate/cmd  |  gate/access
ESP32:      gate_access.ino + config.h, relay GPIO 26
Pi:         main.py + config.py, relay GPIO 17
Verify:     bash scripts/verify-all.sh
```

Next action

1) Run Section 6.2 desk verify. 2) Upload ESP32 sketch (Section 6.3). 3) Follow Section 7 on the real box. See esp32-arduino-controller/README.md for wiring details.